

Introduction

The primary tools in this chapter are Gaussian process inference (§4.3), and the state-space formulation of Gauss–Markov processes (§5), in particular in the algorithmic form of filtering and smoothing (Algorithms 5.1 and 5.2).

► 9.1 Motivation

Integration is a significant numerical problem in many fields of science and engineering. It is a key step in inference, where it is encountered when averaging over the many states of the world consistent with observed data. Indeed, a provocative Bayesian view is that integration is the single challenge separating us from systems that fully automate statistics. More speculatively still, such systems may even exhibit *artificial intelligence* (AI).¹

But integration is also an operation with long history, and elementary enough to be covered at early levels of mathematical education. As such, integration provides the ideal starting point for a presentation of the intuitions driving Probabilistic Numerics and how they might be used to derive practical probabilistic numerical algorithms. This chapter will study how numerical integration – also known as *quadrature* – can be formulated as inference.

The results presented here expand and detail observations from a 1988 paper by Persi Diaconis (1988), and touch on related results by Anthony O’Hagan (1991).

As a running example, we will consider the univariate² function

$$f : \mathbb{R} \rightarrow \mathbb{R}^+, \quad f(x) := \exp\left(-(\sin(3x))^2 - x^2\right) \quad (9.1)$$

¹ M. Hutter. *Universal Artificial Intelligence*. Texts in Theoretical Computer Science. 2010.

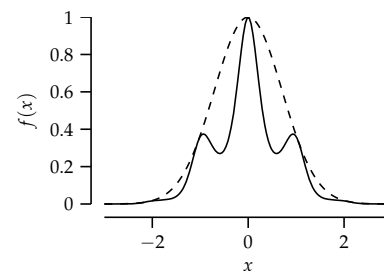


Figure 9.1: The smooth function $f(x)$ and an upper bound, the Gaussian function.

² We first focus on the univariate case for its pedagogical simplicity. In fact, analogously to multivariate GP regression, all below Bayesian quadrature methods extend directly to the d -dimensional case for all $d \in \mathbb{N}$. Note, however, that they – at the time of writing – are only practical up to $d \lesssim 20$ dimensions. See §10.1.2 for more details.

(Figure 9.1). The handful of symbols on paper that make up Eq. (9.1) fully specify a unique *deterministic* function. For arbitrary double precision values of $x \in \mathbb{R}$, a laptop computer can evaluate $f(x)$ to machine precision in a few nanoseconds, using only multiplication and addition, and the “atomic” functions $\exp$ and $\sin$, which are part of elementary programming language definitions, like the GNU C library.³ Repeated evaluations at the same value x will always return the same result. There is nothing *imprecise* about Eq. (9.1) and thus, one may argue that there is also nothing *uncertain* about the function f . However, the definite real number

$$F := \int_{-3}^3 f(x) dx \in \mathbb{R} \quad (9.2)$$

cannot be computed straightforwardly or elementarily. Since f is clearly integrable, there is one and only one correct value of F . But this real number cannot be found in standard tables of analytic integrals.⁴ And there is no atomic operation in low-level libraries providing its value to machine precision. Despite the formal clarity of Eq. (9.1), we are evidently *uncertain* about the value of F , because we cannot provide a correct value for it without further work. This is *epistemic* uncertainty, the kind arising from a lack of knowledge.

But it is easy to constrain the value of F to a finite domain: Because $f(x)$ is strictly positive, $F > 0$. A second glance at Eq. (9.1) lets us notice that f is bounded above by the Gaussian function:

$$f(x) \leq g(x) := \exp(-x^2) \quad \forall x \in \mathbb{R}.$$

Thus,

$$0 < F < \int_{-\infty}^{\infty} g(x) dx = \sqrt{\pi}.$$

Hence it is possible to define a proper prior measure over the value of F , for example the uniform measure $p(F) = \mathbb{U}_{(0, \sqrt{\pi})}(F)$. Thus, if we can collect “data” Y that is related to F through some correctly defined and sufficiently regular likelihood function $p(Y | F)$, then the resulting posterior $p(F | Y)$ will converge towards the true value F , at an asymptotically optimal rate for this likelihood.⁵ The question is thus, which prior, and which likelihood, should we choose?

► 9.2 Monte Carlo

Before introducing a probabilistic formulation of integration, it is helpful to briefly consider a popular *non-probabilistic*, but

³ www.gnu.org/software/libc. There is a fuzzy boundary between what constitutes an “atomic” operation and a numerical algorithm. We will be pragmatic and define the `libc` as the set of atomic operations. A more abstract notion, in line with the purposes of this text, albeit also less concrete, is as follows: consider an algorithm a , a map $a(\theta) = \hat{\omega}$ taking inputs $\theta \in \Pi$ from some space Π that define a computational task with a true (potentially unknown) solution ω , and returning an estimate $\hat{\omega}$. If there are values of θ that the algorithm accepts without throwing an error, such that the resulting $\hat{\omega}$ deviates from the true ω by more than machine precision, then we might call a a *numerical* method, otherwise a low-level, atomic routine. Put succinctly, a numerical method produces an estimate that may be off, while an atomic routine just returns correct numbers. Hence, numerical methods can benefit from a non-trivial notion of uncertainty, while in atomic routines the associated uncertainty is always nil. This is not meant to be a perfectly rigorous definition of the term, and it is imperfect (for example, numerical methods also often have precision parameters and budgets to tune and spend, an aspect ignored by this definition), but a precise definition is not needed in practice anyway.

⁴ Gradshteyn and Ryzhik (2007)

Exercise 9.1 (inspirational, see discussion in §9.3). *Even without appealing to the Gaussian integral, we could also bound f from above with the unit function $u(x) = 1$ on the integration domain, and would arrive at the looser bounds $0 < F < 6$. On the other hand, if we allow the use of the function erf (which is also in `glibc`), we could refine the upper bound to the definite integral over g , arriving at $0 < F < \sqrt{\pi} \operatorname{erf}(3)$. In which sense is this more “precise” prior more “correct”? Is there a most correct, or optimal prior? There is no immediate answer to these questions, but it is helpful to ponder it while reading the remainder of this chapter.*

⁵ Le Cam (1973)

stochastic, way to compute integrals. That is, an approach based on the use of random numbers. Methods that compute using summary statistics of random numbers are known as *Monte Carlo (MC) methods*. They are a mainstay of computational statistics.

Assume the task is to compute $F = \int_a^b f(x) dx$, for some generic integrable function f . Consider a probability measure $p(x)$ over the domain (a, b) that has the following properties:

1. $p(x) > 0$ wherever $f(x) \neq 0$ in (a, b) ; and
2. it is possible to draw random samples from p at acceptable computational cost (that is, using a source of uniform random numbers and a small number of atomic operations).

For the example integrand f with the integration limits $(a, b) = (-3, 3)$ from Eq. (9.1), the uniform measure $p(x) = \mathbb{U}_{-3,3}(x)$ fulfils these properties. Another possibility is the Gaussian measure restricted to $[-3, 3]$, that is,

$$p(x) = \begin{cases} \frac{1}{\text{erf}(3)\sqrt{\pi}} \exp(-x^2) & \text{if } |x| \leq 3, \\ 0 & \text{else.} \end{cases} \quad (9.3)$$

Draws from Eq. (9.3) could, for example, be constructed relatively efficiently by *rejection sampling*.⁶

Using N i.i.d. samples $x_i \sim p(x)$, $i = 1, \dots, N$ from any such p , we can construct the *importance sampling* estimator

$$\hat{F} := \frac{1}{N} \sum_{i=1}^N w(x_i), \quad (9.4)$$

using the function $w(x) := f(x)/p(x)$, which is well-defined by our above assumptions on $p(x)$. The logic of this procedure is depicted in Figure 9.2.

Note that, in contrast to F , the estimator $\hat{F}$ is a *random* number. By constructing $\hat{F}$, we have turned the problem (9.1) – inferring an uncertain but unique deterministic number – into a stochastic, statistical one. This introduction of external stochasticity introduces a different form of uncertainty, the *aleatory* kind, a lack of knowledge arising from randomness (more in §12.3). Because we have full control over the nature of the random numbers, it is possible to offer a quite precise statistical analysis of $\hat{F}$:

Lemma 9.2. *If F is integrable, the estimator $\hat{F}$ is unbiased. Its variance is*

$$\text{var}(\hat{F}) = \frac{1}{N} \text{var}_p(w), \quad (9.5)$$

⁶ Robert and Casella (2013), §2.3

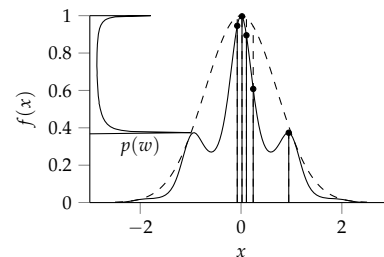


Figure 9.2: Monte Carlo integration. Samples are drawn from the Gaussian measure $p(x)$ (unnormalised measure as dashed line, samples as black dots), and the ratio $w(x) = f(x)/p(x)$ evaluated for each sample. The histogram plotted vertically on the left (arbitrary scale) shows the resulting distribution $p(w) = \int w(x) dp(x)$. Its expected value, times the normalisation $\text{erf}(3)\sqrt{\pi}$, is the true integral. Its standard deviation determines the scale for convergence of the Monte Carlo estimate.

assuming $\text{var}_p(w)$ exists. Hence, the standard deviation (the square root of the variance, which is a measure of expected error) drops as $\mathcal{O}(N^{-1/2})$ – the convergence rate of Monte Carlo integration.

This is a strong statement given the simplicity of the algorithm it analyses: random numbers from almost any measure allow estimating the integral over *any integrable function*. This integrator is “good” in the sense that it is unbiased, and its error drops at a known rate.⁷ The algorithm is also relatively cheap: it involves drawing N random numbers, evaluating $f(x)$ once for each sample, and summing over the results. Given all these strong properties, it is no surprise that Monte Carlo methods have become a standard tool for integration. However, there is a price for this simplicity and generality: as we will see in the next section, the $\mathcal{O}(N^{-1/2})$ convergence rate is far from the best possible rate. In fact, we will find ourselves arguing that it is the *worst* possible convergence rate amongst sensible integration algorithms.

Monte Carlo is not limited to problems where samples can be drawn exactly. Where exact sampling from a distribution is difficult, Monte Carlo is often practically realised through *Markov-Chain Monte Carlo* (MCMC). These iterative methods do not generally achieve the $\mathcal{O}(N^{-1/2})$ convergence rate, but they can still be shown to be *consistent*, meaning that their estimate of the integral asymptotically converges to its true value.

► 9.3 Relevance and Limitations of Prior Knowledge

The Monte Carlo method returns a “frequentist” statistical estimator: a random number whose distribution has known good properties. The strength of this approach is that the analysis of Lemma 9.2 requires only few assumptions. In particular, it involves almost no restrictive requirements on the integrand f . In practice, however, it is not necessary to be so cautious, for we invariably *do* know quite a lot about the integrand f – after all, the computer performing the computation must have access to a definition of f in a formal language: the source code encoding f . That description is bound to reveal additional information about the integrand. In our running example, we can inspect Eq. (9.1) to see that it is bounded above and below, smooth, etc.

How much performance can be gained if the algorithm is allowed *some* assumptions about the integrand – some *prior information*? This chapter introduces a probabilistic class of algorithms for integration, known as *Bayesian quadrature* (BQ).⁸ We will discover that some classical integration rules, in partic-

Proof of Lemma 9.2. $\hat{F}$ is unbiased because its expected value is

$$\langle \hat{F} \rangle = \frac{1}{N} \sum_i \int_a^b w(x_i) p(x_i) dx_i = F,$$

given that the draws x_i are i.i.d., and assuming that the $w(\cdot)$ function is known (more on this later). As $\hat{F}$ is a linear combination of i.i.d. random variables, the variance of $\hat{F}$ is immediately a linear combination of the respective variances:

$$\text{var}(\hat{F}) = \frac{\sum_i \text{var}_p(w_i)}{N^2} = \frac{\text{var}_p(w)}{N}.$$

□

⁷ The multiplicative constant $\text{var}_p(w)$ can even be estimated at runtime! Albeit usually not without bias. See also Exercise 9.3 for some caveats.

Exercise 9.3 (moderate, solution on p. 361). One of the few assumptions in Lemma 9.2 is the existence of $\text{var}_p(w)$. Try to find an example of a simple pair of integrand f and measure p for which this assumption is violated.

⁸ See O’Hagan (1991). The term “Bayesian quadrature” is used ambiguously to mean a broad class of methods based on the idea of integration as probabilistic inference, or to refer to O’Hagan’s specific algorithm, or something in between, for example any quadrature rule derived from a Gaussian process prior. We use it in the first, general sense.

ular the fundamental *trapezoidal rule*, arise naturally from this perspective.

Constructing a probabilistic quadrature method requires two ingredients. Together they form a general recipe that will also feature for all other probabilistic numerical methods:

1. A **model** describing the relationship between the *latent* object of interest and the *computable* quantities. That is, a joint probability measure $\mathcal{M} := p(F, Y)$ over the integral F and a data set $Y := [f(x_1), \dots, f(x_N)]$ of function values. Assuming sufficient regularity of $\mathcal{M}$, the product rule (2.1) allows to write $\mathcal{M}$ as a *generative model* in terms of a prior $p(F)$ and a likelihood $p(Y | F)$ for F . Such a generative model will usually involve the integrand itself as a latent variable, because (as Y consists of evaluations of f) F and Y are independent when conditioned on f :

$$\begin{aligned} p(F, Y) &= p(F) p(Y | F) = \int p(F | f) p(Y | F, f) dp(f) \\ &= \int p(F | f) p(Y | f) dp(f). \end{aligned}$$

Thus, the posterior on F can be computed via the posterior on f :

$$p(F | Y) = \int p(F | f) dp(f | Y).$$

The model thus encodes *assumptions*, not just over the integrand, but also over its relationship to the numbers being computed to estimate it.

2. A **design rule** governing the choice of nodes $X := [x_1, \dots, x_N]$ where f is evaluated by the computer. In general, such design rules will be a function mapping the model $\mathcal{M}$, the previous choices $\{x_j\}$ and previously collected data $Y_{<i} := \{y_j := f(x_j) \mid j < i\}$ to the decision x_i . If the rule explicitly includes previous evaluations $Y_{<i}$, it will be called *adaptive* or *closed-loop*. If X is chosen solely on the model, the model is called *non-adaptive*, or *open-loop*. A non-adaptive design may be less sample-efficient, but can also have lower computational cost, as the design can be pre-computed. We will discuss both non-adaptive and adaptive approaches in §10.2. The role of the design rule is thus to translate the information encoded in the model – and, for adaptive rules, the collected observations – into actions of the agent.

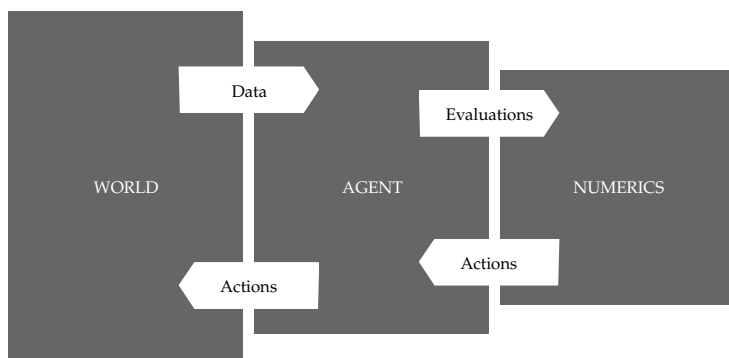


Figure 9.3: An agent, and its model of the world, must necessarily be simpler and smaller than the world itself. Similarly, a numerical algorithm, and its model of the problem, must be simpler and smaller than the problem itself.

It is tempting to try and construct the model $\mathcal{M}$ by encoding as much valid prior information as possible in a probability measure. But this philosophical desire for a maximally informative prior must be balanced against practical constraints. We are looking for a probability measure $\mathcal{M}$ that not only puts high mass close to the truth, but also allows the *efficient computation* of a posterior $p(F | Y)$, using only atomic operations.

A potential criticism of the probabilistic viewpoint on computation is that there is really only one acceptable prior – the one that puts unit probability mass on the true solution on the task, thus expresses zero uncertainty. But such a “perfect”, or “informed”, prior would be just as analytically intractable as the numerical task itself, and thus of no practical concern. Asking for a probabilistic numerical method using the perfect prior is analogous to asking for a classical method that returns the correct answer without performing a single computation, or to posit that a learning machine is only “well-posed” if it does not require any training data. The goal must be to construct *tractable* priors that have various desirable properties. For instance, these priors should give rise to non-trivial uncertainty at intermediate points in the computation. Figure 9.3 illustrates the point: models must be simpler than the problem modelled.